

Week 3 Preview — Floats, Approximation, and Bisection Search

What we'll learn this week

Computers store numbers in binary, so some decimals like 0.1 can only be stored **approximately** — tiny errors that grow when repeated. So instead of demanding exact answers, we find "close enough" ones. Then we meet a much faster way to search: like a lecturer finding a student in an alphabetically-seated hall by asking the middle person's name and dismissing half the room each time — that is bisection search.

Terms to know

Term	One-line definition
floating point	How Python stores real numbers: a pair of integers (significant digits and a base-2 exponent) — an approximation
binary representation	Writing numbers as sums of powers of 2 using only bits 0 and 1, which is what computer hardware stores
epsilon	The small allowed distance to the true answer: stop when $ \text{guess}^2 - x < \text{epsilon}$ ("close enough")
approximation method	Like guess-and-check, except we increment the guess by a small float and stop when close enough, since exact is impossible
bisection search	Guess the midpoint of an interval, learn if the answer is higher or lower, and cut the search space in half each time
logarithmic growth	Bisection needs about $\log_2(N)$ steps versus N steps for one-by-one search — 30 guesses instead of 23 million for 54321

Try this code before class

Taken from Lecture 5, "Implementation" of the approximation method (slide 26). **Predict the output before you run it.**

```
x = 36
epsilon = 0.01
num_guesses = 0
guess = 0.0
increment = 0.0001
while abs(guess**2 - x) >= epsilon:
    guess += increment
    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```

Will it print exactly 6? Roughly how many guesses do you expect? (The lecture notes report about 60,000 — why so many?)

Questions to think about

1. "Never use `==` to test floats." Where else in life do we accept "close enough" instead of exact equality, and how do we choose the tolerance (our epsilon)?
 2. Bisection search only works when the answers have an *ordering* and you get feedback like "too high / too low." Think of a guessing situation where you get no such feedback — why does halving no longer help there?
-

Source: MIT OCW 6.100L *Introduction to CS and Programming Using Python* (Fall 2022), Lectures 5–6 (Ana Bell). Licensed under CC BY-NC-SA. <https://ocw.mit.edu>